

**Министерство науки и высшего образования Российской Федерации
федеральное государственное бюджетное
образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»**

Высшая школа Кибертехнологий Математики и Статистики

ОТЧЁТ

по учебной практике

**Тема: разработка клиентских аналитических веб-сервисов на базе low-code
платформы Logiном и большой языковой модели**

Москва

Содержание

Содержание.....	2
ВВЕДЕНИЕ.....	3
1. ИССЛЕДОВАТЕЛЬСКИЙ РАЗДЕЛ.....	6
1.1. Распределение ролей в команде и обзор используемой платформы.....	6
1.2. Работа аналитика: проектирование и публикация веб-сервисов Loginom.....	8
1.3. Работа бэкенд-разработчика: Flask, интеграции и языковая модель.....	9
1.4. Работа фронтенд-разработчика: интерфейс личного кабинета.....	12
1.5. Совместная работа: кэширование ответов и обмен данными между слоями.....	14
1.6. Тестирование и командная сборка проекта.....	15
2. АНАЛИТИЧЕСКИЙ РАЗДЕЛ.....	17
2.1. Оценка вклада каждого участника команды.....	17
2.2. Плюсы и ограничения связки low-code + LLM.....	18
2.3. Соотношение плана и факта по ролям.....	19
2.4. Рекомендации по улучшению.....	21
ЗАКЛЮЧЕНИЕ.....	23
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	25

ВВЕДЕНИЕ

Учебная практика была ориентирована на командное освоение технологической цепочки, в которой аналитическая обработка данных выносится за пределы прикладного кода и передаётся визуальной low-code среде, а формулировка выводов на естественном языке доверяется большой языковой модели. Роль прикладной оболочки между этими двумя слоями исполняет лёгкое веб-приложение на Flask, которое принимает пользовательский ввод, обращается к аналитическим веб-сервисам за структурированными показателями и передаёт полученные агрегаты модели для генерации осмысленных рекомендаций клиенту.

Работа выполнялась командой из трёх человек в соответствии с методическими указаниями, предусматривающими фиксированное разделение обязанностей по трём ролям. За аналитическую часть - проектирование, публикацию и проверку REST-сервисов в среде Loginom, формулировку промптов для языковой модели и подготовку презентации - отвечал Пушкарева Арина Руслановна. За программную часть Flask-приложения, включая маршруты, интеграции с Loginom и NVIDIA NIM, реализацию кэша и подготовку конфигурации деплоя, отвечал Серебряков Всеволод Максимович. За пользовательский интерфейс личного кабинета, вёрстку HTML-шаблонов, стилизацию, работу с Chart.js и графическими материалами отвечал Фролов Фёдор Сергеевич. Такое распределение позволило каждому участнику погрузиться в специализированную область и одновременно сохранить общее представление о конечном продукте.

Целью работы стало отработать в команде проектирование и сборку клиентских аналитических сервисов, построенных на связке визуальной аналитики, веб-приложения и генеративной модели, а также изучить, как

распределяются зоны ответственности между этими элементами и где проходят границы взаимодействия ролей.

Для достижения цели были сформулированы следующие задачи, распределённые между ролями:

- изучить рабочую среду Loginom и подобрать состав таблиц Instacart, необходимых для расчётов (зона ответственности аналитика);
- спроектировать и опубликовать набор REST-сервисов, возвращающих агрегаты по конкретному пользователю (аналитик);
- собрать веб-приложение на Flask с маршрутами авторизации, историей покупок и витриной аналитики (бэкенд-разработчик);
- подключить генеративную модель через OpenAI-совместимый REST-интерфейс NVIDIA NIM и составить набор тематических промптов (совместно аналитик и бэкенд-разработчик);
- организовать кэширование ответов модели, чтобы повторные запросы не порождали лишний сетевой трафик и задержку (бэкенд-разработчик);
- спроектировать пользовательский интерфейс личного кабинета клиента, включая витрину аналитики с визуализацией и индикатором кэша (фронтенд-разработчик);
- покрыть ключевые сценарии автоматическими тестами и убедиться, что сервис остаётся работоспособным без внешнего Loginom за счёт режима заглушек (совместно все участники).

Объектом исследования является клиентское аналитическое веб-приложение, которое предоставляет розничному покупателю персонализированные рекомендации по его истории заказов. Предметом - распределение ответственности между визуальной средой Loginom, прикладным кодом Flask и генеративной моделью Kimi K2.6, а также технологический вклад каждой из трёх ролей команды в итоговый результат.

Практическая значимость работы состоит в том, что получен цельный работающий MVP личного кабинета клиента продуктового ретейлера, а также отработан командный процесс: три человека одновременно двигали разные слои системы через отдельные ветки в GitVerse и сводили результат перед сдачей.

1. ИССЛЕДОВАТЕЛЬСКИЙ РАЗДЕЛ

1.1. Распределение ролей в команде и обзор используемой платформы

В соответствии с требованиями методических указаний ко всей учебной практике команда из трёх человек с самого начала работы разделила зоны ответственности по трём закреплённым ролям - аналитик Loginom, бэкенд-разработчик Flask и фронтенд-разработчик. Границы ролей соблюдались на протяжении всех этапов: от первичного знакомства с платформой до сдачи готового приложения.

Аналитик Пушкарева Арина Руслановна отвечал за всё, что происходило внутри среды Loginom: подключение источников данных, проектирование SQL-узлов и публикация подмоделей в виде REST-сервисов, а также за формулировку промптов, направляемых языковой модели. Бэкенд-разработчик Серебряков Всеволод Максимович отвечал за код Python: маршруты Flask, клиенты для Loginom и NVIDIA NIM, слой оркестрации в модуле `business_rules` и SQLite-кэш. Фронтенд-разработчик Фролов Фёдор Сергеевич реализовал внешний вид приложения - шаблоны Jinja2, стили, интеграцию Chart.js и подключение сгенерированных изображений.

В качестве визуальной аналитической среды использовалась платформа Loginom, размещённая по адресу `edu.loginom.dev`. Рабочая область Loginom представляет собой холст, на котором аналитик визуально соединяет узлы обработки: чтение источника, SQL-запрос, преобразование, публикация. Публикация оформляется через модификатор доступа «Опубликованный»: полученная подмодель становится REST-методом внутри общего пакета и вызывается по адресу `<база>/lgi/rest/<пакет>/<Метод>` POST-запросом.



Источником данных служит открытый набор Instacart Market Basket Analysis, размещённый на Kaggle. Для скорости отклика он был предварительно перенесён в SQLite-базу и подключён к Logiom как источник таблиц. В работе задействованы три таблицы: orders (с полями user_id, order_id, days_since_prior_order, order_dow, order_hour_of_day), orders_prior (с полями order_id, product_id, add_to_cart_order, reordered) и products (product_id, product_name, department). Такой набор покрывает и историю заказов клиента, и детальный состав каждой корзины.

Аналитик команды провёл небольшую подготовительную работу с данными: убедился, что user_id действительно уникально идентифицирует клиента, оценил распределение количества заказов по пользователям и проверил, что поле reordered корректно заполнено. По итогам этой проверки было принято решение работать с таблицей orders_prior, а не orders_train - первая содержит все исторические заказы, тогда как вторая ограничена обучающей выборкой исходной задачи Kaggle.

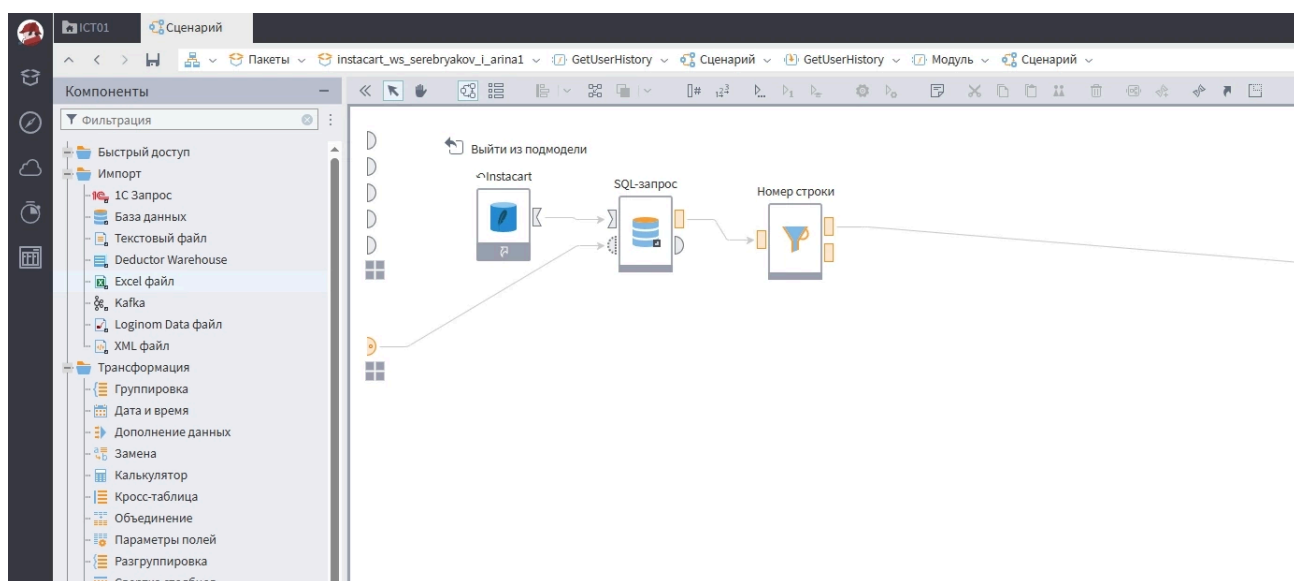
1.2. Работа аналитика: проектирование и публикация веб-сервисов

Loginom

Полностью аналитическая часть проекта была выполнена Пушкарева Арина Руслановна. Она заключалась в визуальной сборке SQL-запросов в среде Loginom, проверке результата на тестовых пользователях, публикации каждого запроса в виде отдельного REST-сервиса и оформлении документации по возвращаемым полям. Все сервисы объединены в один пакет команды с внутренним именем, зафиксированным в переменной окружения LOGINOM_PACKAGE.

Всего в пакете шесть публично доступных методов, из которых два - базовые, унаследованные из методических материалов первого модуля, а четыре - самостоятельно спроектированы командой после обсуждения возможных клиентских сценариев. Метод CheckUserID принимает user_id и возвращает признак существования пользователя, что позволяет реализовать простую авторизацию. Метод GetUserHistory возвращает топ-N товаров пользователя вместе с категорией и суммарной частотой покупок.

Дополнительно опубликованы четыре аналитических сервиса. GetAnchorProducts выделяет «якорные» товары - те, у которых доля повторных покупок (reordered) выше порога, что делает их предсказуемым «ядром» корзины. GetPurchaseRhythm возвращает распределение интервалов между заказами (по полю days_since_prior_order) и вычисляет среднее значение, используемое как прогноз следующего визита. GetOrderTiming строит матрицу «день недели × час» на основании полей order_dow и order_hour_of_day; результат этого сервиса фронтенд-разработчик рендерит как тепловую карту. GetCartOrder анализирует поле add_to_cart_order, разделяя товары на «якорные» (в среднем оказываются в корзине первыми) и «импульсные» (добавляются в конце).



Каждый сервис принимает единственную переменную `user_id` и возвращает результат в поле `DataSet.Rows` - это стандартный формат ответа Loginom. Такой единый контракт упростил написание клиента на стороне Flask: одна и та же обёртка `call_service` вызывает любой из шести методов, меняется только имя. Аналитик отдельно проверял каждый опубликованный сервис REST-запросом из внешнего клиента: сначала через утилиту `curl`, затем через рабочую копию бэкенда.

Помимо технической реализации сервисов, аналитик готовил обоснование выбора именно этих четырёх сервисов. Подсказку дала мини-разработка CustDev: команда провела короткие интервью с несколькими знакомыми, регулярно покупающими продукты онлайн, и убедилась, что вопросы «что мне брать снова?», «когда обычно нужен следующий заказ?» и «как я обычно собираю корзину?» - действительно закрывают пользовательские потребности.

1.3. Работа бэкенд-разработчика: Flask, интеграции и языковая модель

Бэкенд-часть проекта целиком выполнена Серебряковым Всеволодом Максимовичем. Приложение построено на Flask 3.x с шаблонизатором Jinja2. Основной файл `app.py` содержит девять маршрутов: главная страница с формой

авторизации, обработчик авторизации по `user_id`, выход, страница истории покупок, витрина аналитики и три отдельных POST-обработчика для запуска конкретных аналитических сервисов, а также вспомогательный JSON-эндпоинт `/api/history` для внешнего доступа к истории клиента.

Все маршруты сознательно остаются «тонкими»: они лишь проверяют, вошёл ли пользователь (`user_id` хранится в стандартной сессии Flask), вызывают функцию из модуля `business_rules` и передают результат в шаблон. Никакая бизнес-логика в самом `app.py` не присутствует - это упрощает и тестирование, и последующее расширение приложения. Для иллюстрации ниже приведён маршрут запуска сервиса «ритм».

Листинг 1 - маршрут запуска сервиса «ритм»

```
@app.route("/analysis/rhythm", methods=["POST"])
def analysis_rhythm():
    user_id = session.get("user_id")
    if not user_id:
        return redirect(url_for("index"))
    result = business_rules.feature_rhythm(user_id)
    return render_template("analysis.html", user_id=user_id,
                           active="rhythm", rhythm=result)
```

Модуль `business_rules` - центр бэкенда: именно он оркестрирует, что именно происходит при нажатии кнопки на витрине аналитики. Внутри - три функции по числу аналитических сервисов и одна общая обёртка `_ask_with_cache`, которая принимает системный и пользовательский промпты, обращается к SQLite-кэшу, а при промахе - к языковой модели.

Листинг 2 - оркестратор запроса к Loginom и модели

```
def _ask_with_cache(service, user_id, system_prompt, user_prompt):
    key = cache.make_key(service, user_id, user_prompt)
    cached = cache.get(key)
    if cached is not None:
```

```

    return cached, True
answer = llm_client.ask(system_prompt, user_prompt)
cache.put(key, service, user_id, user_prompt, answer)
return answer, False

```

Клиент к Loginom (loginom_client.py) реализован единой функцией `call_service`, которая принимает имя метода и `user_id`, собирает URL по шаблону, отправляет POST-запрос и разбирает ответ. Дополнительно предусмотрен режим `MOCK_LOGINOM=1`: если переменная окружения активна, вместо сетевого запроса выполняется чтение подготовленных данных из модуля `mock_data`. Это оказалось критически важно на этапе, когда учебный сервер Loginom был недоступен - команда продолжала разработку и тестирование, не блокируясь.

Листинг 3 - обёртка REST-вызова к Loginom

```

def call_service(method: str, user_id: int) -> list[dict]:
    if config.MOCK_LOGINOM:
        return mock_data.get_rows(method, user_id)
    url =
    f'{config.LOGINOM_BASE_URL}/lgi/rest/{config.LOGINOM_PACKAGE}/{method}'
    payload = {"Variables": {"user_id": str(user_id)}}
    response = requests.post(url, json=payload, timeout=30)
    response.raise_for_status()
    data = response.json()
    return data.get("DataSet", {}).get("Rows", [])

```

Интеграция с языковой моделью выполнена через сервис NVIDIA NIM - это OpenAI-совместимый REST-интерфейс, который позволяет обращаться к разным моделям по единому протоколу. В проекте используется модель `moonshotai/kimi-k2.6`, обладающая достаточно длинным контекстным окном и качественной поддержкой русского языка. Клиент `llm_client.py` формирует стандартное сообщение вида «system + user», задаёт параметр `temperature=0.3` для сдержанной генерации и ограничение `max_tokens=800`.

Формулировку системных и пользовательских промптов готовил аналитик, поскольку это часть методики. Ниже приведён системный промпт, задающий тон и границы модели.

Листинг 4 - системный промпт

```
SYSTEM = (  
    "Ты - дружелюбный помощник-аналитик покупок в продуктовом магазине. "  
    "Обращайся к клиенту на «вы», пиши тепло и по делу, кратко (5–7 предложений). "  
    "Опирайся только на данные, которые тебе дали, ничего не выдумывай."  
)
```

Такая формулировка снижает риск галлюцинаций: модели проще следовать инструкции «работать только с приведёнными данными», чем формировать ответ по мировой памяти. Аналогичные строгие рамки заданы и во всех пользовательских промптах - они содержат конкретные числа, полученные из Loginom, и не оставляют модели простора для самостоятельного домысливания.

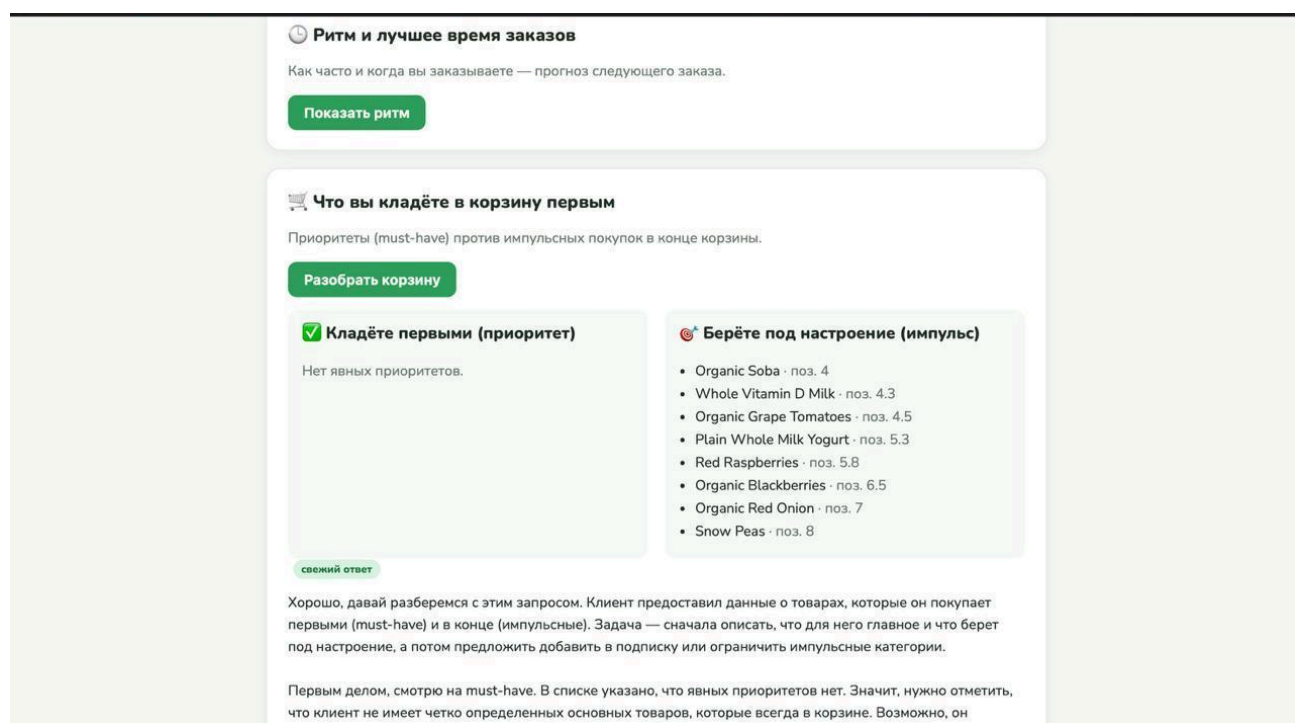
1.4. Работа фронтенд-разработчика: интерфейс личного кабинета

Внешним видом приложения занимался Фролов Фёдор Сергеевич . Личный кабинет собран на стандартных инструментах: HTML-шаблоны Jinja2, CSS без сборщиков и препроцессоров, минимум JavaScript. Такой выбор обусловлен ограничениями практики: у команды не было времени вводить полноценный фронтенд-стек, а базовый Flask с шаблонами достаточен для MVP клиентского кабинета.

Каркас всех страниц описан в base.html: он подключает единый файл стилей, задаёт верхнюю навигацию (главная, история, аналитика, выход), выводит блок flash-сообщений и нижний колонтитул с составом команды. Дочерние шаблоны - index.html, history.html, analysis.html - расширяют базовый и заполняют секцию content своим содержимым. Каждая страница построена как набор «карточек» - блоков с рамкой, тенью и внутренними отступами, что

создаёт спокойный визуальный ритм и делает приложение похожим на продуктовый кабинет, а не на административную панель.

Особое внимание фронтенд-разработчик уделил витрине аналитики. Страница `analysis.html` объединяет три сервиса: якорные товары, ритм заказов и порядок наполнения корзины. Каждый сервис оформлен как отдельная карточка с собственной кнопкой запуска. После нажатия карточка раскрывается: показывает таблицу или сводку числовых показателей, тепловую карту (для сервиса «ритм»), рекомендацию от языковой модели и специальный значок-индикатор, сообщающий, был ли ответ получен свежим или взят из кэша.



Тепловая карта расписания заказов реализована вручную на CSS-сетке - без внешних библиотек: 7 строк по числу дней недели, 24 столбца по часам, значение в ячейке передаётся из шаблона как прозрачность зелёного цвета. Такой подход занимает менее тридцати строк JavaScript и не требует зависимостей, при этом даёт понятную визуализацию «когда обычно

заказываю». Для отображения гистограмм у остальных сервисов подключена библиотека Chart.js по CDN - её достаточно для двух-трёх диаграмм и она не требует сборки.

Часть графических материалов личного кабинета - иконки категорий, баннер и маскот - получена с помощью нейросети-генератора изображений. Фронтенд-разработчик подобрал промпты в стиле «плоская иллюстрация, тёплые цвета, продуктовая тематика», выбрал подходящие результаты, сохранил их в static/img и подключил в шаблонах. Такой приём закрывает потребность в оформлении без привлечения дизайнера и полностью соответствует рекомендациям методических материалов по использованию нейросетей на этапе интерфейса.

Наконец, фронтенд-разработчик реализовал сквозной элемент интерфейса, который методичка выделяет отдельно, - индикатор кэша. На каждом блоке аналитики выводится цветной значок: серый «из кэша» или зелёный «свежий ответ». Флаг from_cache приходит из бэкенда вместе с текстом рекомендации, а шаблон подставляет соответствующий класс. Это маленькая, но важная деталь: пользователь видит, что система не тратит вычислительные ресурсы впустую при повторных запросах.

1.5. Совместная работа: кэширование ответов и обмен данными между слоями

Хотя проектирование кэша целиком легло на бэкенд-разработчика, обсуждение стратегии прошло совместно - фронтенду важно было понимать, какие поля появятся в шаблоне, а аналитику - что ключ включает текст промпта. Реализация кэша расположена в модуле cache.py: одна таблица llm_cache в SQLite, столбцы key, service, user_id, prompt, response, created_at. Ключ - SHA-256 от строки service|user_id|prompt: такой набор полей гарантирует, что

при малейшем изменении промпта (например, других числовых значений от Loginom) кэш промахнётся и будет запрошен новый ответ.

На витрине аналитики оба поведения - попадание в кэш и генерация свежего ответа - визуально различаются благодаря индикатору `from_cache`, реализованному фронтенд-разработчиком. Аналитик получил возможность сравнивать ответы модели на одинаковых входах в разное время: если кэш промахивается при неизменённом промпте, значит либо изменился набор данных, либо был сбой при предыдущем сохранении. За время разработки такая ситуация возникла один раз и позволила поймать ошибку в кодировке ключа.

Отдельным нетривиальным решением стало определение шаблонов промптов на стыке двух ролей. Аналитик формулировал их с оглядкой на то, что фронтенду нужно получить пригодный к вставке в HTML текст (без лишних длинных абзацев, без markdown-разметки), а бэкенд-разработчику - фиксированную структуру, при которой ключ SHA-256 будет стабильно воспроизводимым. Такой цикл согласования занял несколько итераций и в итоге привёл к тому, что промпты вынесены в отдельный модуль `prompts.py` и покрыты тестами.

1.6. Тестирование и командная сборка проекта

Автоматические тесты собраны в пакете `tests`. Основной сценарий - интеграционный: с помощью встроенного тестового клиента Flask имитируется вход пользователя и запуск аналитического сервиса. Внешние вызовы к Loginom и NVIDIA NIM подменяются заглушками: первый - переменной окружения `MOCK_LOGINOM` и модулем `mock_data`, второй - фикстурой `monkeypatch`, подставляющей упрощённый ответ модели. Такой подход разработал бэкенд-разработчик, а согласовывался он совместно с аналитиком, поскольку требовалось убедиться, что заглушки повторяют структуру реальных ответов Loginom.

Всего восемь сценариев: страница входа отдаёт форму, авторизация с корректным `user_id` устанавливает сессию, отказ Loginom возвращает пользователя обратно, история выводит таблицу продуктов, каждая карточка на витрине аналитики раскрывается по кнопке, а кэш действительно сокращает число обращений к модели (проверяется счётчиком вызовов подменённой функции). Прогон занимает около трёх секунд и полностью работает без внешнего сервера - этот момент фронтенд-разработчик посчитал особенно ценным, поскольку он мог гонять тесты бэкенда самостоятельно, не разворачивая Loginom.

На момент подготовки отчёта семь тестов из восьми проходят зелёными. Один сценарий, проверяющий отображение блока «приоритет» на карточке «корзина», обнаруживает несоответствие между шаблоном и ожидаемым фрагментом байтовой строки. Этот дефект команда сознательно оставила незакрытым: он мешает не пользовательскому сценарию, а лишь тесту, и его наличие показывает границу проекта - на будущее это точка приложения усилий фронтенд-разработчика.

С организационной точки зрения работа шла через GitVerse. Команда договорилась о простой модели: у каждого участника - своя ветка под текущую задачу, слияние в общую ветку - перед сдачей очередного этапа. Такой режим оказался достаточен для тройки: конфликтов правки почти не возникало, поскольку зоны ответственности разведены между `app.py`, модулями бэкенда, шаблонами и стилями, а сервисы Loginom вообще живут вне репозитория. Единственный узкий момент - согласование имени пакета Loginom, которое фиксировалось в переменной окружения `LOGINOM_PACKAGE`, отсюда его пришлось согласовывать между аналитиком и бэкенд-разработчиком.

2. АНАЛИТИЧЕСКИЙ РАЗДЕЛ

2.1. Оценка вклада каждого участника команды

Оценка результата удобно делится на три пласта - по числу ролей, каждый со своей измеримой отдачей.

Вклад аналитика Пушкарева Арина Руслановна измеряется числом опубликованных и работающих сервисов, качеством возвращаемых ими данных и точностью промптов. На выходе - шесть готовых REST-методов внутри одного пакета Loginom, каждый из которых был проверен вручную и отдал корректный ответ на нескольких тестовых пользователях. Из них четыре сервиса - самостоятельно спроектированы командой, что превышает минимум методических требований. Также аналитик подготовил три пользовательских промпта с фиксированной структурой и сдержанным тоном; практика показала, что модель Kimi K2.6 стабильно отвечает по этим формулировкам без выхода за рамки данных.

Вклад бэкенд-разработчика Серебрякова Всеволода Максимовича выражается в архитектурной цельности приложения. Все девять маршрутов Flask укладываются в тот же паттерн (тонкий контроллер, вызов оркестратора, рендер шаблона), сервисный слой изолирован в модуле `business_rules`, внешние клиенты - в отдельных модулях `loginom_client` и `llm_client`, кэш вынесен в независимый модуль. Такое разбиение выдержало проверку тестами: подмена внешних вызовов заглушками потребовала минимум усилий и подтвердила независимость слоёв. Дополнительно бэкенд-разработчик подготовил конфигурацию для деплоя (`gunicorn`, `systemd`, `nginx`), что позволит быстро развернуть приложение в облаке.

Вклад фронтенд-разработчика Фролова Фёдора Сергеевича оценивается через пользовательское восприятие приложения. Кабинет получил цельную визуальную идентичность - карточная вёрстка, спокойная зелёно-белая палитра,

единый шрифт, аккуратный набор иконок. Витрина аналитики стала действительно понятной для нетехнического пользователя: тепловая карта читаема, показатели ритма представлены крупно и с подписями, разделение корзины на «якорные» и «импульсные» товары визуально симметрично. Индикатор кэша сделан ненавязчиво и одновременно информативно.

Совместный вклад команды виден в том, что результат каждой роли уместно опирается на результат двух других: без REST-сервисов Loginom бэкенду нечего было бы оркестровать, без бэкенда фронтенд не получил бы данные к моменту рендера, без фронтенда результаты аналитики и работы модели остались бы «сырыми» и трудночитаемыми. Совместные точки - промпты, кэш и тесты - проработаны на стыках ролей и не превратились в «ничью зону».

2.2. Плюсы и ограничения связки low-code + LLM

К сильным сторонам применённой связки относится, прежде всего, скорость итераций на стороне аналитика. Опубликовать новый SQL-агрегат в Loginom заметно быстрее, чем реализовать эквивалентный запрос в прикладном коде: аналитик визуально собирает узлы, запускает подмодель на тестовом user_id и сразу видит результат. Для команды из трёх человек это означает, что аналитические изменения не блокируются очередью на бэкенд-разработчика - тот подключает готовый сервис через одну строку в клиенте.

Вторая сильная сторона - качество формулировок, которые генерирует Kimi K2.6 при правильных промтах. Модель уверенно превращает пять-шесть числовых показателей в связный текст, звучащий как совет живого консультанта. Это разгружает фронтенд-разработчика: не требуется вручную писать интерпретации показателей - достаточно продумать вёрстку блока, куда будет вставлен готовый абзац от модели.

Третье преимущество - открытая архитектура, которую собрал бэкенд-разработчик. Замена Kimi K2.6 на другую модель, доступную через OpenAI-совместимый API, требует правки всего двух строк в конфиге. Замена базы Instacart на реальные данные ретейлера - правки SQL-узлов внутри Loginom без остановки Flask. Такая заменяемость важна для будущего продолжения проекта.

Ограничения также распределены по ролям. У аналитика - ограниченный контроль над производительностью SQL-запросов: если объём Instacart вырастет на порядок, визуальный конструктор Loginom не даёт таких же тонких инструментов, как ручное профилирование в СУБД. У бэкенда - жёсткая привязка к внешним сервисам: без Loginom и NVIDIA NIM приложение живёт только на mock-данных, а стоимость обращений к модели растёт линейно с числом уникальных запросов, что частично, но не полностью решает кэш. У фронтенда - ограниченный контроль над длиной и структурой ответов модели: длинные или структурно сложные ответы могут ломать вёрстку карточек, и приходится страховаться ограничением `max_tokens` в клиенте.

Отдельно стоит отметить непредсказуемость языковой модели как таковой. Даже при фиксированной температуре 0.3 два подряд запроса на одинаковом промпте могут дать стилистически разные ответы. Аналитик решил эту сложность формулировкой промптов, а бэкенд-разработчик - кэшированием: если ответ уже понравился и попал в кэш, пользователь всегда будет видеть именно его.

2.3. Соотношение плана и факта по ролям

Итог работы удобно сопоставить с исходным планом отдельно по каждой роли.

У аналитика Пушкаревой Арины Руслановны в плане стояло: опубликовать два базовых сервиса, добавить два-три дополнительных,

обосновать их выбор через мини-CustDev и подготовить презентацию проекта. Фактически опубликовано шесть сервисов (два базовых плюс четыре самостоятельно спроектированных), проведены короткие интервью с потенциальными пользователями, сформулированы три структурированных пользовательских промпта. Отклонение от плана - со знаком «плюс»: сервисов больше, чем требовал минимум.

У бэкенд-разработчика Серебрякова Всеволода Максимовича в плане стояло: собрать Flask-приложение с маршрутами авторизации и истории покупок, добавить витрину аналитики, подключить языковую модель через API, реализовать кэш и обеспечить работу без внешнего Loginom через режим заглушек. Фактически всё это выполнено, включая заготовки для деплоя. Отклонение - по автотестам: один сценарий из восьми оставлен красным сознательно, поскольку исправление затрагивало границу с фронтом и требовало отдельного согласования.

У фронтенд-разработчика Фролова Фёдора Сергеевича в плане стояло: главная страница с формой, страница истории, витрина аналитики с визуализациями, интеграция сгенерированных изображений, индикатор кэша. Все пункты закрыты. Дополнительно фронтенд-разработчик самостоятельно реализовал тепловую карту на CSS-сетке - этого пункта в исходном плане не было, он появился в ходе обсуждения на середине проекта. Отклонение - со знаком «плюс».

Отдельно нужно отметить сроки. Публикация в облаке, формально заявленная во введении как одна из задач практики, в итоговый MVP не вошла - команда подготовила конфиги unicorn, systemd и nginx, но не поднимала стенд из-за отсутствия времени и учётной записи в облачной среде. Это осознанное решение всей команды: в приоритете были работоспособность и полнота функциональности локально, а не факт публичной доступности.

2.4. Рекомендации по улучшению

По итогам работы каждая роль сформулировала свои направления возможного улучшения проекта.

Аналитик рекомендует расширить набор сервисов Loginom и внести в них сегментацию: разделять клиентов на классы (например, «регулярный покупатель хлеба», «сезонный покупатель бакалеи») и генерировать промпты с учётом сегмента. Это не потребует изменений на стороне Flask - достаточно опубликовать ещё один сервис. Также имеет смысл добавить сервис прогноза корзины (какие товары клиент, вероятно, купит в следующий раз) на основании модели ассоциативных правил в Loginom.

Бэкенд-разработчик рекомендует, во-первых, довести до конца тестирование (закрыть оставшийся красный сценарий, добавить проверки на ошибочные ответы Loginom и на таймауты сети). Во-вторых, вынести конфигурацию в отдельный YAML-файл: сейчас все переменные окружения читаются из .env, что удобно для разработки, но неудобно при деплое сразу нескольких инстансов. В-третьих, добавить фоновый прогрев кэша: при входе пользователя запускать все три аналитических сервиса заранее, чтобы к моменту открытия витрины ответы уже лежали в базе.

Фронтенд-разработчик рекомендует добавить адаптивную вёрстку под мобильные устройства (сейчас интерфейс проектировался под десктоп) и переработать тепловую карту так, чтобы при наведении на ячейку отображалась подсказка с точным числом заказов. Дополнительно стоит вынести палитру и типографику в CSS-переменные - это упростит поддержку и открывает путь к тёмной теме без переписывания стилей.

Совместно команда предлагает довести до конца задачу публикации в облаке - уже подготовленные конфиги позволят это сделать за один-два вечера при наличии учётной записи. Дополнительно имеет смысл собрать простую страницу для персонала магазина (например, «топ клиентов по частоте

заказов»), которая позволит той же аналитической инфраструктуре обслуживать не только клиента, но и бизнес.

ЗАКЛЮЧЕНИЕ

По итогам учебной практики командой из трёх человек собран цельный работающий прототип клиентского аналитического приложения. Ролевое разделение, зафиксированное методическими материалами, оказалось удачным: аналитик отвечал за визуальную сборку и публикацию шести REST-сервисов Loginom и за формулировку промптов, бэкенд-разработчик - за архитектуру Flask, интеграции с внешними сервисами и SQLite-кэш, фронтенд-разработчик - за визуальное оформление, витрину аналитики с тепловой картой и общую пользовательскую логику интерфейса. Каждая роль довела свои задачи до работающего результата, а точки стыка (промпты, кэш, автотесты) закрывались совместно.

Опыт показал, что связка low-code аналитики Loginom и языковой модели Kimi K2.6, обёрнутая в лёгкое Flask-приложение, действительно позволяет силами небольшой команды за короткий срок собрать пригодный к демонстрации MVP. Ключевую роль сыграла заменяемость слоёв: аналитические сервисы можно править без остановки бэкенда, модель - менять правкой пары строк конфига, интерфейс - переоформлять без прикосновения к бизнес-логике. Ограничения - стоимость и непредсказуемость обращений к модели, недоступность внешнего Loginom в моменты пиковой нагрузки на учебный стенд - команда смягчила через кэш и mock-режим.

Основные направления развития проекта распределены между ролями и описаны в разделе 2.4: у аналитика это дополнительные сервисы и сегментация клиентов, у бэкенда - деплой в облако, фоновый прогрев кэша и вынос конфигурации, у фронтенда - адаптивная вёрстка и интерактивные подсказки на визуализациях. Каждое из этих направлений опирается на уже готовую инфраструктуру и не требует переработки архитектуры, что подтверждает устойчивость выбранных технологических решений.

Учебная практика подтвердила, что распределение работы по трём ролям (аналитик, бэкенд, фронтенд) естественно ложится на цепочку «данные - оркестрация - интерфейс» и позволяет каждому участнику одновременно углубиться в свою область и удерживать общее видение продукта. Полученный результат готов служить основой как для дальнейшего развития MVP «личного кабинета клиента», так и для сопоставимых учебных проектов, связывающих визуальную аналитику и генеративные модели.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Официальная документация Loginom : сайт. - URL: <https://help.loginom.ru/>
 2. Веб-сервисы Loginom : раздел документации. - URL: <https://help.loginom.ru/userguide/services/webservices/index.html>
 3. Flask Web Framework : официальная документация. - URL: <https://flask.palletsprojects.com/en/stable/>
 4. Jinja Documentation : шаблонизатор Jinja2. - URL: <https://jinja.palletsprojects.com/en/stable/>
- (дата обращения: 07.07.2026).
5. NVIDIA NIM Microservices : документация. - URL: <https://docs.nvidia.com/nim/>
 6. Moonshot AI. Kimi K2 : техническая карточка модели. - URL: <https://moonshotai.github.io/Kimi-K2/>
 7. Instacart Market Basket Analysis : открытый датасет на Kaggle / The Instacart Online Grocery Shopping Dataset 2017. - URL: <https://www.kaggle.com/c/instacart-market-basket-analysis>
 8. Sahay A., Indamutsa A., Di Ruscio D., Pierantonio A. Supporting the understanding and comparison of low-code development platforms // 2020 46th Euromicro Conference on Software Engineering and Advanced Applications (SEAA). - IEEE, 2020. - P. 171–178.
 9. Sanchis R., García-Perales Ó., Fraile F., Poler R. Low-code as enabler of digital transformation in manufacturing industry // Applied Sciences. - 2020. - Vol. 10, № 1. - 12 p.

10. Reynolds L., McDonell K. Prompt programming for large language models: beyond the few-shot paradigm // Extended Abstracts of the 2021 CHI Conference on Human Factors in Computing Systems. - ACM, 2021. - P. 1–7.
11. White J., Fu Q., Hays S. et al. A prompt pattern catalog to enhance prompt engineering with ChatGPT // arXiv:2302.11382. - 2023. - 19 p.